DOCUMENTO TÉCNICO · IDEATHON FATEC ARAÇATUBA

Maturidade Técnica em Tempo Limitado: Arquitetura e Segurança para MVP de Caronas com Supabase

Especificação Tecnológica, Modelo de Validação Institucional e Engenharia de Software

1. STACKS ESCOLHIDAS E JUSTIFICATIVAS TÉCNICAS

Front-End: HTML, CSS & JavaScript
Justificativa: A escolha destas tecnologias deve-se ao fato de serem os padrões da web, garantindo que o projeto seja executado de forma rápida, eficiente e compatível com qualquer navegador ou dispositivo.

Back-End: Node.js
Justificativa: Foi selecionado pela facilidade de integração com o restante da aplicação e pelo ecossistema robusto.

Banco de Dados & BaaS: Supabase
Justificativa: Escolhido pela praticidade e agilidade na configuração, permitindo estruturar e conectar o banco de dados ao back-end de maneira simples e rápida.

2. VALIDAÇÃO INSTITUCIONAL E SEGURANÇA DE ENTRADA

A criação de um sistema de caronas exige validação inicial rigorosa. A restrição a domínios institucionais (`@fatec.sp.gov.br` e `@aluno.cps.sp.gov.br`) atua como primeira linha de defesa. Para evitar contornos na camada de aplicação, a verificação é imutável no banco de dados.

Gatilho PostgreSQL com SECURITY DEFINER
A solução utiliza um trigger disparado antes de cada inserção (`BEFORE INSERT ON auth.users`). A função executa com privilégios elevados de superusuário (`SECURITY DEFINER`) para consultar a tabela restrita `public.allowed_email_domains` antes mesmo da gravação definitiva do usuário.

- **Validação Normalizada:** Extrai o domínio do e-mail, converte para minúsculas e valida contra a lista de permissões. Se o domínio for inválido, lança uma exceção e aborta a transação.
- **Sincronização de Perfil:** Após o aceite, um gatilho secundário cria automaticamente o registro na tabela `public.profiles` definindo a flag de verificação institucional como verdadeira.

3. COMPONENTES DA ARQUITETURA E INFRAESTRUTURA

Componente	Papel no Sistema	Justificativa Técnica & Segurança
public.allowed_email_domains	Lista de domínios autorizados.	Gestão dinâmica de domínios sem alterações no código-fonte. Protegida por RLS.
Função Trigger (PL/pgSQL)	Intercepta e valida o e-mail.	Executa sob `SECURITY DEFINER` para acessar dados antes do usuário ter token válido.
public.profiles	Metadados e flag `is_verified`.	Relaciona-se com `auth.users` via FK. Possui RLS focado no proprietário do registro.
Node.js / Express	API intermediária / Middleware.	Valida a assinatura do JWT via JWKS e evita uso da chave `service_role` para não burlar o RLS.
Supabase GoTrue & PostgREST	Serviço de Auth e API RESTful.	Emite JWTs assinados e converte consultas SQL em endpoints protegidos pelo filtro PostgreSQL.

4. ARQUITETURA DINÂMICA: FLUXO DE SOLICITAÇÃO DE CARONA

O fluxo de interação dinâmica segue um modelo de validação em duas etapas (Backend Node.js + Supabase DB):

- **1. Requisição Front-End:** O cliente envia um `POST /api/rides/request` com o cabeçalho `Authorization: Bearer <JWT>`.
- **2. Middleware Express:** Valida a expiração, emissor (`iss`) e público (`aud`) do token usando as chaves públicas (JWKS) do Supabase. Rejeita com HTTP 401 se inválido.
- **3. Persistência via PostgREST:** O backend repassa o JWT ao Supabase. O banco aplica a regra RLS e confirma a gravação apenas se `passenger_id = auth.uid()`.

5. SEGURANÇA EM NÍVEL DE LINHA (ROW LEVEL SECURITY - RLS)

O Supabase expõe a base de dados via API RESTful PostgREST. A RLS atua como uma **camada de segurança fundamental no nível de banco de dados (security floor)**, garantindo que o acesso a dados sensíveis seja restrito mesmo que a chave anônima pública seja conhecida.

> **Diferença Crucial: USING vs. WITH CHECK**
> **Cláusula USING:** Filtra quais linhas existentes podem ser LIDAS, ATUALIZADAS ou EXCLUÍDAS (filtro de leitura/escrita existente).
> **Cláusula WITH CHECK:** Valida os NOVOS DADOS que estão sendo gravados ou modificados (filtro em INSERT e UPDATE).

6. MATRIZ DE POLÍTICAS RLS PRIORITÁRIAS

Tabela	Operação	Papel (Role)	Cláusula SQL / Condição	Propósito de Segurança
`public.profiles`	SELECT / DELETE	authenticated	`USING (auth.uid() = user_id)`	Garante que o usuário veja e exclua apenas seu próprio perfil.
`public.profiles`	INSERT	authenticated	`WITH CHECK (auth.uid() = user_id)`	Impede a criação de perfis associados a IDs de terceiros.
`public.profiles`	UPDATE	authenticated	`USING (...) WITH CHECK (...)`	Edita o próprio perfil e previne a alteração da posse da coluna `user_id`.
`public.ride_requests`	ALL (CRUD)	authenticated	`auth.uid() = passenger_id`	Isola as solicitações de carona estritamente ao passageiro proprietário.
`public.ride_requests`	SELECT	anon, auth	`USING (status = 'AVAILABLE')`	Exibe publicamente apenas corridas ativas sem dados sensíveis.

7. DESEMPENHO E DEFESA EM PROFUNDIDADE

- **Indexação B-tree:** Todas as colunas consultadas em RLS (`user_id` , `passenger_id`) exigem índices. A ausência reduz drasticamente a performance para varreduras sequenciais.
- **Otimização por Subconsulta:** Envolver `(SELECT auth.uid())` instrui o PostgreSQL a executar a função uma única vez por consulta, reutilizando o cache.
- **Gestão Segura de Chaves:** A chave `SUPABASE_SERVICE_ROLE_KEY` ignora o RLS e não deve ser usada no backend para rotas comuns. O backend repassa o JWT usando a `SUPABASE_ANON_KEY`.
- **Permissões de Coluna:** A RLS protege linhas, não colunas. Dados sensíveis (ex: CPF) exigem restrições diretas com `GRANT UPDATE (coluna) ON TABLE`.

> **Conclusão Executiva**
> A escolha da stack (HTML/CSS/JS, Node.js e Supabase) aliada à arquitetura de segurança no banco de dados oferece a velocidade exigida pelo Ideathon sem comprometer a maturidade técnica da solução.